Hydration Node PR 1071

Security Review

Cantina Managed review by:

Joran Honig, Lead Security Researcher

Frank Castle, Security Researcher

Jakub Heba, Associate Security Researcher

June 28, 2025

Contents

1 Introduction **2**
- 1.1 About Cantina . . . 2
- 1.2 Disclaimer . . . 2
- 1.3 Risk assessment . . . 2
  - 1.3.1 Severity Classification . . . 2

2 Security Review Summary **3**

3 Findings **4**
- 3.1 High Risk . . . 4
  - 3.1.1 Max buy price check performs comparison of prices denominated in different currencies . . . 4
- 3.2 Medium Risk . . . 4
  - 3.2.1 Risks related to peg selection . . . 4
  - 3.2.2 HSM potentially violates collateralization invariant of Hollar . . . 6
- 3.3 Low Risk . . . 6
  - 3.3.1 Collateral Removal Can Be DoS'ed via Minimal Transfer . . . 6
  - 3.3.2 The imbalance and buy back limit are manipulable . . . 7
  - 3.3.3 Arbitrage might be blocked by facilitator minting limits . . . 7
  - 3.3.4 Arbitrage bot operates on potentially outdated values . . . 7
  - 3.3.5 Prevent accepting the Hollar asset as collateral . . . 8
- 3.4 Informational . . . 8
  - 3.4.1 Avoid doing manual complex type conversions . . . 8
  - 3.4.2 Missing collateral assets parameters bounds validation . . . 9
  - 3.4.3 Duplication of peg normalization code . . . 9
  - 3.4.4 Duplication of logic with slight deviation in impelementation . . . 9

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity	Description
Critical	*Must* fix as soon as possible (if already deployed).
High	Leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
Medium	Global losses <10% or losses to only a subset of users, but still unacceptable.
Low	Losses will be annoying but bearable. Applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.
Gas Optimization	Suggestions around gas saving practices.
Informational	Suggestions around best practices or readability.

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Hydration is a next-gen DeFi protocol which is designed to bring an ocean of liquidity to Polkadot.

From May 7th to May 10th the Cantina team conducted a review of hydration-node on commit hash 6b5d5cf2. The team identified a total of **12** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	1	1	0
Medium Risk	2	0	2
Low Risk	5	2	3
Gas Optimizations	0	0	0
Informational	4	3	1
Total	**12**	**6**	**6**

3 Findings

3.1 High Risk

3.1.1 Max buy price check performs comparison of prices denominated in different currencies

Severity: High Risk

Context: lib.rs#L973-L998

Description: In `do_trade_hollar_in` (lib.rs#L976-L998) the HSM performs a check comparing the simulated execution price of a trade to a maximum price derived from the peg and a governance configurable `max_buy_price_coefficient`. However, the denomination currency for the execution price, and max buy price do not match resulting in a faulty check. `buy_price` is derived from the execution price and denominated in hollar, while `max_buy_price` is derived from `peg` which is sourced from the stableswap module and denominated in collateral:

```
// hsm/src/lib.rs:976
let execution_price = (sim_collateral_amount, sim_hollar_amount);

// 4. Calculate final buy price with fee
let buy_price = math::calculate_buy_price_with_fee(execution_price, collateral_info.buy_back_fee)
    .ok_or(ArithmeticError::Overflow)?;

// %. Calculate final swap amounts
let (final_hollar_amount, final_collateral_amount) =
    calculate_final_amounts((sim_hollar_amount, sim_collateral_amount), buy_price)?;

log::trace!(target: "hsm", "Hollar amount {:?}, buyback limit {:?}", final_hollar_amount, buyback_limit);
// Check if the requested amount exceeds the buyback limit
ensure!(
    HollarAmountReceived::<T>::get(collateral_asset).saturating_add(final_hollar_amount) <= buyback_limit,
    Error::<T>::MaxBuyBackExceeded
);

// 6. Calculate max price
let max_price = math::calculate_max_buy_price(peg, collateral_info.max_buy_price_coefficient);
ensure!(
    math::ensure_max_price(buy_price, max_price),
    Error::<T>::MaxBuyPriceExceeded
);
```

For reference `calculate_max_buy_price` performs a $(a/b) < (c/d) \implies (ad < cb)$ conversion to then compare the resulting values as follows:

```
    let buy_price_check = U128::from(buy_price.0).full_mul(U128::from(max_price.1));
    let max_price_check = U128::from(buy_price.1).full_mul(U128::from(max_price.0));
    buy_price_check <= max_price_check
```

Recommendation: Ensure that prices are denominated in the same currency during comparison. Note that other operations, such as those determining the imbalance, do rely on the current peg denomination.

Hydration: Fixed in PR 1102

Cantina Managed: Fix verified.

3.2 Medium Risk

3.2.1 Risks related to peg selection

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Description: The HSM pallet is designed to introduce incentives to maintain a stable value between the hollar token and selected collateral tokens. Its mechanisms have two inputs: a peg price, and the price of a stableswap pool at the beginning of a block. The HSM code is generally agnostic of the type of peg used, for this reason we'll explore the principal options that might be used:

- Static peg.

- Dynamic peg through historic derivation.

The HSM introduces two incentives:

- It will always sell Hollar at the peg (charging some fees).
- It will buy some Hollar above the spot price while the spot price is below the peg.

This first incentive is a strong feedback mechanism that ensures the Hollar price never rises above the value of any collateral. It also means that if any collateral spot price diverges below its peg that Hollar will mirror that price movement.

The second incentive is a feedback mechanism for which its relative strength can be configured through various parameters (including: `buy_back_fee` and `buy_back_rate`). Unlike the previous mechanism it works over a longer time as the `buy_back_rate` limits how much Hollar can be sold to the HSM per block.

How these mechanics affect the Hollar system and its stability is dependent on the type of peg used.

- Static Peg: Using a static peg value (e.g. 1 HOLLAR/USDT) effectively implies that the HSM executes a long strategy on its collateral tokens.

  The HSM provides an arbitrage opportunity (up to the Gho facilitator minting limit) allowing users to purchase Gho using depegged stable coins as if they never lost their value. This in turn will reduce the value of Hollar.

  This can amount to a significant loss if the collateral token never regains its peg. However, overall damage might be limited if the collateral re-acquires its peg. The principal losses being the incentives that the HSM gives out to try to re-acquire its peg with respect to other collaterals.

  - Recommendation: You've already introduced a parameter "`max_in_holding`" which serves as a partial mitigation which can suffice depending on the risk model. Similarly, the limit on minting for the facilitator account provides a hard limit on the impact of a depeg.

  Finally, limiting velocity (limiting the amount of tokens that can enter the HSM per block) could further improve resilience of the HSM.

- Dynamic Peg: Using a dynamic peg based on historic prices of Hollar/collateral can introduce unique risks, primarily it creates the possibility that Hollar value permanently drifts with respect to the dollar.

  The problem arises when the value of a collateral token drops below its intended peg before the dynamic peg value has adjusted to the new price. Strong `do_trade_hollar_out` incentives will cause Hollar to follow the downward price movement of the depegged collateral token leading to Hollar adjusting its peg.

  Consider a scenario where we have multiple uncorrelated collateral tokens and one of the collateral tokens suddenly loses its peg going from 1 USD to 0.90 USD. The stableswap peg value is 1/1 and its spot price, due to arbitrage, is at 0.90/1.

  1. The `do_trade_hollar_out` incentives (which prevent Hollar from growing in value w.r.t. collateral) will allow users to trade 0.90 cents of collateral for roughly 1 USD worth of Hollar (minus a fee).
  2. Arbitrageurs should quickly close this arbitrage opportunity until Hollar is roughly equal in value to the depegged collateral.

  At this point Hollar has reaquired its peg with respect to the collateral token, but lost its peg with respect to USD and the other collateral tokens.

  3. The system will start using its `do_trade_hollar_in` incentives to try and bring up the price of Hollar.

  However, the `do_trade_hollar_out` incentives should keep the Hollar value at 0.90 USD.

  4. After some period (this depends on the stableswap peg configuration) the peg of Hollar and other collaterals readjusts to what equates to 0.90 USD = 1 Hollar.

  Now let's look at what happens when the collateral token repegs:

  5. Arbitrageurs will ensure that all stableswap pools return to a state where all collateral tokens are worth 1 USD.

At this point we have a 1-1 peg with the token that has reaquired its peg, and what equates to a 0.90 USD peg w.r.t. the other collateral tokens.

6. Since the 1-1 collateral is now worth more than Hollar we'll see that the system will implement its `do_trade_hollar_in` incentives to try and bring the price of Hollar back to 1 USD. However, the system will also implement its `do_trade_hollar_out` incentives to keep the value of Hollar at what equates to a 0.90 USD peg with respect to the other tokens.
7. The `do_trade_hollar_out` incentives will keep Hollar at 0.90 USD until it establishes what would be equal to a 0.90 USD peg with respect to the collateral token that has now re-aquired its peg.

- Peg Drift: It's also possible that the combination of incentives and a dynamic peg could introduce a slow upward peg drift.

  The incentives create a model where the expected Price of Hollar will be between the peg and the price at which Hollar can be bought (fee adjusted). If a historically derived peg is used then we should expect the peg to drift upwards.

  It's unclear whether this would happen in practice as the `do_trade_hollar_in` incentives are proportional to the imbalance, and thus likely small while the price of Hollar fluctuates around the peg.

- Recommendation: There are various approaches that might be incorporated in a solution:
  * Incorporating a consensus oracle to provide collateral / usd prices could help prevent erroneously readjusting the Hollar with respect to the stable collateral.
  * Adjusting the mechanics, such as by introducing some type of rate / velocity limit on the `do_trade_hollar_out` incentives, can help avoid following downward price movements while the dynamic peg has not yet readjusted.

It's likely worthwhile to validate the effectiveness and soundness of the chosen adjustments and final configuration using simulations.

Hydration: Acknowledged.

Cantina Managed: Acknowledged.

3.2.2 HSM potentially violates collateralization invariant of Hollar

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Description: Users that stake tokens to borrow Hollar will be required to maintain a healthy margin allowing the Hollar to be overcollateralized, however the HSM has no such requirement. It's possible that the HSM causes Hollar to become undercollateralized when one of the used collateral tokens has a drop in value.

Recommendation: AAVE's documentation on the GHO Stability Module describes a "Last Resort Liquidation" mechanism. Similar functionality could help ensure that Hollar remains overcollateralized.

Hydration: Acknowledged.

Cantina Managed: Acknowledged.

3.3 Low Risk

3.3.1 Collateral Removal Can Be DoS'ed via Minimal Transfer

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: In the `remove_collateral_asset` function (lib.rs#L487), the code checks that there is no remaining balance of the collateral asset in the HSM pallet before allowing its removal:

```
ensure!(amount.is_zero(), Error::<T>::CollateralNotEmpty);
```

This assertion can lead to a DoS vulnerability on the function with minimal cost. Any user can send the minimum transferable amount of the collateral asset to the pallet's wallet, causing the `remove_collateral_asset` call to revert.

The likelihood is high, as this can be done at any time to prevent collateral removal, without needing to interact with the HSM pallet or the trading mechanism.

The impact is low to medium. For example, if the admin has a full list of collateral assets (e.g., 10 assets) and wants to replace one with a more valuable or usable asset, an attacker can prevent the removal by continuously DoS'ing the function.

Recommendation: Instead of requiring the balance to be strictly zero, update the function to automatically transfer any remaining collateral tokens to a trusted address (such as the admin or treasury) before removing the asset. This prevents malicious dusting attacks from blocking collateral removal.

Hydration: Acknowledged.

Cantina Managed: Acknowledged.

3.3.2 The imbalance and buy back limit are manipulable

Severity: Low Risk

Context: lib.rs#L966-L970

Description: The HSM considers the imbalance of tokens in a stable swap pool, and uses the absolute imbalance to determine a max amount of tokens to buy back in a given block. An attacker can trivially increase the amount of tokens that the HSM is willing to buy back by depositing liquidity in the stableswap pool. The impact of this vector is limited as increasing the Imbalance using this method requires lots of liquidity and exposes the attacker to impermanent loss.

Furthermore, the ability to increase the buy back limit on its own does not seem to impose a significant risk to the protocol, provided that the other measures (such as the buy back fee) are configured appropriately.

Recommendation: Consider the fact that an attacker is able to increase the buy back limit in configuring the risk model security parameters of the HSM pallet.

Hydration: Acknowledged.

Cantina Managed: Acknowledged.

3.3.3 Arbitrage might be blocked by facilitator minting limits

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: The `execute_arbitrage()` function performs what's essentially a flash loan, and strictly decreases the amount of minted Hollar. However, its initial `mint()` operation will fail when the facilitator account tied to the HSM has reached it's Hollar minting limit. This should not be necessary since the arbitrage operation strictly decreases the amount of Hollar minted by the facilitator.

Recommendation: Executing arbitrage operations from a different facilitator would allow the arbitrage operation to function successfully even if the HSM facilitator account is at its limit. Furthermore, you could include an invariant check (using an `ensure!` call) to ensure that the total amount minted by the arbitrage facilitator is zero at the end of every arbitrage extrinsic execution.

Hydration: Fixed in PR 1114

Cantina Managed: Fix verified.

3.3.4 Arbitrage bot operates on potentially outdated values

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: The arbitrage bot uses an initial pool snapshot to determine trade values, however trades that precede the arbitrage functions' execution might have already changed the pool state with respect to

the snapshot. This can cause the arbitrage execution to fail. Setting the transaction priority high should be sufficient to prevent front-running by regular transactions, but does not account for DCA trades executed before transactions.

Recommendation: There are multiple ways that interference can be avoided, one might be to incorporate current reserve values in calculating trade values.

Hydration: Acknowledged.

Cantina Managed: Acknowledged.

3.3.5 Prevent accepting the Hollar asset as collateral

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Prevent accepting the Hollar asset as collateral: The `add_collateral_asset` function currently checks to prevent adding duplicate collateral assets with the following code:

```
ensure!(
    !Collaterals::<T>::contains_key(asset_id),
    Error::<T>::AssetAlreadyApproved
);
```

However, the function `ensure_pair` used in the `buy` and `sell` functions does not prevent setting Hollar as both `asset_in` and `asset_out`, which allows users to buy Hollar at a price higher than its actual value:

```
fn ensure_pair(asset_in: T::AssetId, asset_out: T::AssetId) -> DispatchResult {
  ensure!(
    (asset_in == T::HollarId::get() && Self::is_collateral(asset_out))
      || (asset_out == T::HollarId::get() && Self::is_collateral(asset_in)),
    Error::<T>::InvalidAssetPair
  );

  Ok(())
}
```

Moreover, there is no validation to explicitly prevent adding the Hollar asset itself as collateral. This misconfiguration can lead to a serious risk of fund loss for both users and the protocol, especially during buy and sell operations.

Recommendation: Ensure that `asset_id != Hollar ID` when adding a collateral asset.

Hydration: Fixed in PR 1103.

Cantina Managed: Fix verified.

3.4 Informational

3.4.1 Avoid doing manual complex type conversions

Severity: Informational

Context: math.rs#L13, math.rs#L47-L51, math.rs#L60

Description: The HSM module performs various manual conversions of complex types such as conversions between Permill and Ratio. This leads to duplication and increased cognitive complexity.

Recommendation: Consider implementing a canonical `From` trait implementation to convert types such as Permill and Perbill into a Ratio.

Hydration: Fixed in PR 1104.

Cantina Managed: Fix verified.

3.4.2 Missing collateral assets parameters bounds validation

Severity: Informational

Context: lib.rs#L411-L420

Description: In the `add_collateral_asset` and `update_collateral_asset`, configuration of these assets is assigned through parameters such as `purchase_fee` and `buy_back_fee`.

However, there is no validation of them implemented, allowing for whole type range values to be saved in the state. That is problematic in the fees context, as due to the `Permill` usage, fees could be set up to 100%.

Risk of the issue was limited due to the strict slippage checks, which while provided correctly from the caller side - will prevent most of the significant risks, leading to unexpected reverts, rather than financial losses.

Recommendation: We recommend reconsidering implementation of bounded fee parameters, limiting the maximum fee to be below some specific percent, for example 10%.

Hydration: Acknowledged.

Cantina Managed: Acknowledged.

3.4.3 Duplication of peg normalization code

Severity: Informational

Context: lib.rs#L950-L963, lib.rs#L1063-L1076, lib.rs#L1309-L1322

Description: The logic that determines a normalized peg value when no pool source is available is duplicated three times. This introduces a risk when code is refactored as changes need to be duplicated.

Recommendation: Reduce code duplication by introducing functions to execute common operations.

Hydration: Fixed in PR 1104.

Cantina Managed: Fix verified.

3.4.4 Duplication of logic with slight deviation in impelementation

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: The function calculate_arbitrage_opportunity mirrors a lot of computations also performed in do_trade_hollar_in. However, it does so with slight divergences. Primarily it doesn't leverage the clean and well implemented functions in math.rs. The following is an example of semantically equivalent but syntactically divergent code:

- calculate_arbitrage_opportunity.

```
let fee_complement = Permill::from_percent(100).saturating_sub(fee);
```

- math.rs.

```
Permill::one().saturating_sub(buy_back_fee).deconstruct() as u128,
```

Recommendation: Leverage functions from `math.rs` implementation to reduce duplication of operations code.

Hydration: Fixed in PR 1104.

Cantina Managed: Fix verified.